

UNIT 6 · WEEK 9

Signals and shared memory

📅 Week 9 · dates to be confirmed </> Lab TD5 (solved + AI challenge) ~ 3 animations

LAB TD5 · SOLVED

Signals and shared memory (`mmap` , `shm`)

Two ways for processes to communicate without pipes: notifying each other with signals and sharing memory pages. Together they let a process leave a message in memory and "ring the bell" of the other one.

✓ **Lab report**

Hand in this lab with a short LaTeX report using the course template: each exercise explained, the code that matters and what you ran in the terminal. You may use generative AI to write the LaTeX: declare it and check every result. Template, example and Prism guide →

EXERCISES

- | | | | |
|---|---|---|-------------------------------------|
| 1 | Catching SIGUSR1 with <code>sigaction</code> | 4 | Sharing code: a hand-made "library" |
| 2 | Message through <code>mmap</code> + a signal | 5 | System V shared memory |
| 3 | Protections of an anonymous <code>mmap</code> | | |

TD5 · 1

● Solved · tested

Catching SIGUSR1 with `sigaction(2)`

Statement. `mysignal` prints its pid, installs a SIGUSR1 handler that prints the signal number, waits for signals and restores the default handler. Test from another terminal with `kill`: SIGUSR1, the sequence SIGSTOP → SIGUSR1 → SIGCONT, and SIGKILL.

Solution

```
C tds/td5/mysignal.c
```

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
```

```

* ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores
the default action
* From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT
<pid>, kill -KILL <pid>
* After the n-th SIGUSR1 the default action is back: the next SIGUSR1
terminates the process.
*/
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be
interrupted mid-buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n',
0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 ||
write_all(STDERR_FILENO, digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);

    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the
handler runs */
    sa.sa_flags = 0;
    exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

    /* Block SIGUSR1 outside sigsuspend: a signal arriving between the test

```

```

of `received`
    and the wait stays pending instead of being lost (the pause() race).
*/
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and
sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}

```

Real output

```

$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P;
kill -USR1 $P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P;
echo "exit status: $? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)

```

- **With the process stopped** (SIGSTOP), SIGUSR1 is not lost: it stays *pending* and the handler runs on SIGCONT. SIGSTOP and SIGKILL cannot be caught or blocked: they are the administrator's guarantee.
- **The handler only uses async-signal-safe functions** (`write`, not `printf`): the signal can arrive in the middle of a `printf` of the program and re-enter its buffer.
- `sigprocmask` + `sigsuspend` instead of `pause` : if the signal arrives between the test of `received` and `pause`, `pause` would sleep forever. With the signal blocked, it waits pending; `sigsuspend` unblocks it and sleeps in one atomic step.

Animation A6.1 · Signal delivery →

TD5 · 2

● Solved · tested

A message through `mmap` and a signal

Statement. `receiver` maps a 128-byte file, writes its pid at the start and waits for SIGUSR1; when it arrives it prints the message. `sender` maps the same file, reads the pid, copies its standard input into the shared area and sends SIGUSR1. What happens if more than 128 characters are written? How big is the segment really?

Solution

C tds/td5/receiver.c

```
/*
 * TD5 - exercise 2: receiver. Maps block.txt (shared), writes its pid at the
 * start of the
 * mapping, waits for SIGUSR1 and prints the message that the sender left
 * after the pid.
 * ./receiver &      then      echo "hello" | ./sender
 */
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "block.txt"
#define SIZE 128

static void on_usr1(int sig) { (void)sig; } /* only interrupts sigsuspend
*/

int main(void)
{
    struct sigaction sa = { 0 };
    sigset_t usr1, old, wait_mask;
```

```

    /* 1. Block SIGUSR1 BEFORE publishing the pid: a sender can only signal
    us after that,
        and the signal will wait (pending) until sigsuspend. */
    sigemptyset(&usr1);
    sigaddset(&usr1, SIGUSR1);
    exit_if(sigprocmask(SIG_BLOCK, &usr1, &old) == -1, "sigprocmask");
    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);
    exit_if(sigaction(SIGUSR1, &sa, NULL) == -1, "sigaction");

    /* 2. The file must be at least SIZE bytes: mapping past the end of a
    file gives SIGBUS. */
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " FILENAME);
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");          /* the mapping stays
valid */

    /* 3. Publish our pid. */
    pid_t me = getpid();
    memcpy(shared, &me, sizeof me);
    printf("receiver %d: waiting for SIGUSR1\n", me);
    fflush(stdout);

    /* 4. Sleep with SIGUSR1 unblocked, atomically. */
    wait_mask = old;
    sigdelset(&wait_mask, SIGUSR1);
    sigsuspend(&wait_mask);
    exit_if(sigprocmask(SIG_SETMASK, &old, NULL) == -1, "sigprocmask"); /*
SETMASK, not BLOCK */

    shared[SIZE - 1] = '\0';                    /* never trust the
other side */
    printf("received: %s", shared + sizeof me);
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}

```

C tds/td5/sender.c

```
int main(void)
{
    int fd = open(FILENAME, O_RDWR);
    exit_if(fd == -1, "open " FILENAME);
    char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
0);
    exit_if(shared == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    pid_t pid;
    memcpy(&pid, shared, sizeof pid);
    printf("sender: the receiver is %d\n", pid);

    /* Copy stdin into the message area, keeping one byte for '\0'. */
    size_t off = sizeof pid, max = SIZE - 1;
    ssize_t n = 1;
    while (off < max && (n = read(STDIN_FILENO, shared + off, max - off)) >
0)
        off += (size_t)n;
    exit_if(n == -1, "read");
    shared[off] = '\0';

    exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also
push to the file */
    exit_if(kill(pid, SIGUSR1) == -1, "kill");
    exit_if(munmap(shared, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

C tds/td5/mmap_limits.c

```
/*
 * TD5 - exercise 2, last question: how big is the shared segment of a 128-
byte file?
 * The kernel maps whole pages. We map TWO pages of a 128-byte file and write
byte by byte:
```

```

*   bytes 0..127      belong to the file
*   bytes 128..4095   exist (same page) but are never written back to the
file
*   byte 4096         is in a page that lies entirely past the end of the
file: SIGBUS
* The signal is caught and siglongjmp brings us back to report where it
happened.
*/
#include <fcntl.h>
#include <setjmp.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#define FILENAME "limits.txt"
#define FILESIZE 128

static sigjmp_buf env;
static volatile sig_atomic_t caught;
static char *volatile map;          /* volatile: must survive the
siglongjmp */
static volatile long offset;

static void on_fault(int sig)
{
    caught = sig;
    siglongjmp(env, 1);
}

int main(void)
{
    const long page = sysconf(_SC_PAGESIZE);
    int fd = open(FILENAME, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open");
    exit_if(ftruncate(fd, FILESIZE) == -1, "ftruncate");
    map = mmap(NULL, 2 * page, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(map == MAP_FAILED, "mmap");

    struct sigaction sa = { 0 };
    sa.sa_handler = on_fault;
    sigemptyset(&sa.sa_mask);

```

```

    exit_if(sigaction(SIGSEGV, &sa, NULL) == -1 || sigaction(SIGBUS, &sa,
    NULL) == -1, "sigaction");

    if (sigsetjmp(env, 1) == 0)
        for (offset = 0; offset < 2 * page; offset++)
            map[offset] = 'x';

    printf("file size %d bytes, page size %ld bytes, mapping of %ld bytes\n",
    FILESIZE, page, 2 * page);
    printf("wrote bytes 0..%ld; byte %ld raised %s\n", offset - 1, offset,
    caught ? signame(caught) : "nothing");

    exit_if(munmap(map, 2 * page) == -1, "munmap");
    off_t end = lseek(fd, 0, SEEK_END);           /* the file did not
grow */
    exit_if(end == -1, "lseek");
    printf("file size after the writes: %lld bytes\n", (long long)end);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}

```

Real output

```

$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes

```

The real size is one page (4096 bytes). The kernel maps whole pages: bytes 128 to 4095 exist in memory but do not belong to the file (they are never written to disk and the file still has 128 bytes). A page that lies entirely past the end of the file raises **SIGBUS** when touched. That is why `receiver` calls `ftruncate`: mapping an empty file and writing into it also gives SIGBUS.

Fixed

The inherited version restored the mask with `SIG_BLOCK` (it did not restore it: it added to it), called `printf` inside the handler and the message overwrote the published pid. The pid now takes the first bytes and the message comes after it.

TD5 · 3

● Solved · real outputs

Protections of an anonymous `mmap`

Statement. `map_anonymous` asks for one integer with `mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0)`, prints the pointer, reads `*p`, writes 42 and reads it again. Build it with `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` and `PROT_EXEC|PROT_WRITE`, and explain every result with `mmap(2)`.

Solution



C tds/td5/map_anonymous.c

```

/*
 * TD5 - exercise 3: protections of an anonymous mapping.
 * PROT is given at compile time; the Makefile builds map_none, map_read,
map_write, map_exec
 * and map_ew (PROT_EXEC|PROT_WRITE). Each program reads *p, writes 42 and
reads it again.
 * Read mmap(2): on x86-64, PROT_WRITE implies read access; PROT_NONE forbids
everything.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

#include "utils.h"

#ifndef PROT
#define PROT (PROT_READ | PROT_WRITE)
#endif

/* Readable name of the protection flags, e.g. "PROT_EXEC|PROT_WRITE". */
static const char *prot_name(int prot)
{
    static char name[64];
    name[0] = '\0';
    if (prot == PROT_NONE)
        return "PROT_NONE";
    if (prot & PROT_READ) strcat(name, "|PROT_READ");
    if (prot & PROT_WRITE) strcat(name, "|PROT_WRITE");
    if (prot & PROT_EXEC) strcat(name, "|PROT_EXEC");
    return name + 1;
}

int main(void)
{
    int *p = mmap(NULL, sizeof(int), PROT, MAP_PRIVATE | MAP_ANONYMOUS, -1,
0);
    exit_if(p == MAP_FAILED, "mmap");

    fprintf(stderr, "PROT = %s\n", prot_name(PROT));
    fprintf(stderr, "pointer: %p\n", (void *)p);
    fprintf(stderr, "read:    *p = %d\n", *p);          /* anonymous memory

```

```
starts zeroed */
    *p = 42;
    fprintf(stderr, "write:  *p = %d\n", *p);

    exit_if(munmap(p, sizeof(int)) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./map_none
PROT = PROT_NONE
pointer: 0x742c16995000
bash: line 1: 357416 Segmentation fault      (core dumped) ./map_none
[exit status 139]
```

Real output

```
$ ./map_read
PROT = PROT_READ
pointer: 0x7b6b0d393000
read:  *p = 0
bash: line 1: 357431 Segmentation fault      (core dumped) ./map_read
[exit status 139]
```

Real output

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:  *p = 0
write: *p = 42
```

Real output

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Real output

```
$ ./map_ew
PROT = PROT_WRITE|PROT_EXEC
pointer: 0x7872e8140000
read:    *p = 0
write:   *p = 42
```

The MMU enforces the permissions of every page: `PROT_NONE` forbids even reading; `PROT_READ` allows reading (anonymous memory starts at zero) but not writing; on x86-64 `PROT_WRITE` implies read. `PROT_EXEC` alone is an interesting case: on this machine (with memory protection keys) Linux creates *execute-only* memory and the read fails; on processors without that feature the same test reads 0. That is why `mmap(2)` says the effect of each combination "is architecture dependent". Every violation is a page fault that the kernel turns into SIGSEGV (status 139).

TD5 · 4

● Solved · tested

Sharing code: a hand-made "library"

Statement. Simulate a shared library: `load_add` copies the machine code of `add(int, int)` into a 2048-byte file mapped with `MAP_SHARED`; `use_add` maps that file with `PROT_EXEC`, uses it as a function pointer and calls `f(42, 12)`. What happens with several `use_add` at once?

Solution



C tds/td5/load_add.c

```

int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int fd = open(LIBFILE, O_RDWR | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, "open " LIBFILE);
    /* An empty file cannot be mapped usefully (access past its end =
    SIGBUS): give it its size. */
    exit_if(ftruncate(fd, SIZE) == -1, "ftruncate");

    char *code = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    /* Copy the first bytes of add(). Converting a function pointer to void *
    is not ISO C,
       but POSIX requires it to work (dlsym relies on it). */
    memcpy(code, (const void *)add, 64);

    exit_if(munmap(code, SIZE) == -1, "munmap"); /* MAP_SHARED: the bytes
    are in the file */
    return EXIT_SUCCESS;
}

```

C tds/td5/use_add.c

```
int main(void)
{
    int fd = open(LIBFILE, O_RDONLY);
    exit_if(fd == -1, "open " LIBFILE);

    void *code = mmap(NULL, SIZE, PROT_READ | PROT_EXEC, MAP_SHARED, fd, 0);
    exit_if(code == MAP_FAILED, "mmap");
    exit_if(close(fd) == -1, "close");

    int (*f)(int, int) = (int (*)(int, int))code;
    printf("f(42, 12) = %d\n", f(42, 12));

    exit_if(munmap(code, SIZE) == -1, "munmap");
    return EXIT_SUCCESS;
}
```

Real output

```
$ ./load_add && ./use_add
f(42, 12) = 54
```

It works because `add` uses no absolute address: its code is position independent. Several `use_add` share the same physical page (the file's page-cache page), just as every program shares the pages of `libc.so`. A real loader (`ld.so`, `dlopen`) also resolves symbols and relocations. The inherited version mapped the file without giving it a size and ended in SIGBUS.

TD5 · 5

● Solved · tested

System V shared memory

Statement. (1) Create or get a 1024-byte segment with `ftok` + `shmget`, attach it with `shmat`, increment its first integer, print it and detach it with `shmdt`. Run it several times: what do you observe? (2) With `fork` in a 4-round loop, 16 processes register their pid in the segment (number of pids + array of pids) and the first process prints the list. Repeat with 32 processes.

Solution



C tds/td5/shm_counter.c

```

/*
 * TD5 - exercise 5: a System V shared memory segment that survives the
 processes.
 * ./shm_counter increments the first int of the segment and
 prints it
 * ./shm_counter --remove destroys the segment (IPC_RMID)
 * Run it several times: 1, 2, 3... The segment lives in the kernel until it
 is removed or
 * the machine reboots (list them with `ipcs -m`, remove with `ipcrm`).
 * The key comes from ftok(file, id): same file + same id = same key on this
 machine.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#include "utils.h"

#define KEYFILE "/etc/passwd" /* any file that exists on every Linux
 machine */
#define KEYID 'F'
#define SIZE 1024

int main(int argc, char *argv[])
{
    key_t key = ftok(KEYFILE, KEYID);
    exit_if(key == -1, "ftok");
    int id = shmget(key, SIZE, IPC_CREAT | 0644);
    exit_if(id == -1, "shmget");

    if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
        exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl IPC_RMID");
        printf("segment %d removed\n", id);
        return EXIT_SUCCESS;
    }

    int *counter = shmat(id, NULL, 0);
    exit_if(counter == (void *)-1, "shmat");
    counter[0]++; /* a fresh segment is zero-
 filled */

```

```
printf("segment %d (key 0x%08x): counter = %d\n", id, (unsigned)key,
counter[0]);
exit_if(shmdt(counter) == -1, "shmdt");
return EXIT_SUCCESS;
}
```

Real output

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

C tds/td5/shm_fork.c

```
/*
 * TD5 - exercise 5 (end): 16 processes (4 fork() in a loop) register their
 pid in a shared
 * segment: segment[0] = number of pids, then the pids. The original process
 prints the list.
 *
 * "count++; table[count] = pid" is NOT atomic: two processes can read the
 same count and
 * one registration is lost. Build with -DNO_ATOMIC (Makefile target
 shm_race) to see it;
 * the default version reserves the slot with an atomic fetch-and-add (a
 preview of unit 7).
 * The segment is IPC_PRIVATE: only this process and its descendants (which
 inherit the
 * attachment through fork) can see it.
 */
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define FORKS 4 /* 2^4 = 16 processes */
#define SIZE 4096
```

```

int main(int argc, char *argv[])
{
    int forks = argc > 1 ? atoi(argv[1]) : FORKS;
    int id = shmget(IPC_PRIVATE, SIZE, IPC_CREAT | 0600);
    exit_if(id == -1, "shmget");
    int *seg = shmat(id, NULL, 0);
    exit_if(seg == (void *)-1, "shmat");
    /* Mark for deletion now: it disappears when the last process detaches
    (even on a crash). */
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl");

    pid_t original = getpid();
    int children = 0;
    for (int i = 0; i < forks; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            children = 0;          /* the child starts with no children
of its own */
        else
            children++;
    }

#ifdef NO_ATOMIC
    int slot = seg[0] + 1;          /* read ... */
    sched_yield();                 /* ... (widen the window) ... */
    seg[0] = slot;                 /* ... write: not atomic */
#else
    int slot = __atomic_add_fetch(&seg[0], 1, __ATOMIC_SEQ_CST); /* read-
modify-write in one step */
#endif
    if (slot < SIZE / (int)sizeof(int))
        seg[slot] = getpid();

    while (children-- > 0)
        exit_if(wait(NULL) == -1, "wait");

    if (getpid() == original) {
        printf("%d processes registered (expected %d):", seg[0], 1 << forks);
        for (int i = 1; i <= seg[0] && i < SIZE / (int)sizeof(int); i++)
            printf(" %d", seg[i]);
        printf("\n");
    }
    exit_if(shmdt(seg) == -1, "shmdt");
}

```

```
return EXIT_SUCCESS;
}
```

Real output

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568
357571 357569 357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Real output

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

- **The segment outlives the processes:** the counter goes 1, 2, 3... because the segment lives in the kernel until someone deletes it (`IPC_RMID` , `ipcrm`) or the machine reboots. `ipcs -m` shows it. The inherited version deleted it on the first run and the phenomenon was invisible.
- **With 32 processes registrations are lost** (between 16 and 25 out of 32 in the five real runs above): `seg[0]++` is read–add–write, and two processes can read the same value. It is a *race condition*, the topic of unit 7. The solution uses an atomic increment of the processor (`__atomic_add_fetch`).
- `ftok` uses a file that exists on every Linux (`/etc/passwd`); the original statement used `/etc/bashrc` , which does not exist on Ubuntu.

Checklist

Generative AI challenge · Replace ``pause`` with ``sigsuspend``... and prove the race ^

Goal. Write with the AI a program that loses a signal with ``pause()`` (the classic race), measure how often it happens and show that ``sigprocmask`` + ``sigsuspend`` removes it.

Starting prompt. Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

Prompt

I want to demonstrate the `pause()` race experimentally. Help me write: (1) a receiver that does ``while (!arrived) pause();`` with a handler that sets `arrived = 1`; (2) a sender that sends `SIGUSR1` right after the receiver publishes its pid; (3) a script that repeats the experiment 1000 times with a 1 s limit and counts how often the receiver sleeps forever. Then let us write the version with `sigprocmask` + `sigsuspend` and compare. Explain the exact window in which the signal is lost.

How to check the result

- The ``pause()`` version hangs at least a few times (if it never does, widen the window with a ``usleep`` before ``pause``).
- The ``sigsuspend`` version never hangs in 1000 runs.
- You can point at the exact line between the test and the wait.

Declare your use of AI: tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.